HELIOS

Sistema unificado de citas medicas

Sprint 1 - Confirmacion de cita

MVP funcional en produccion - 20 bugs resueltos - 5 agentes IA

Grupo HELIOS

Jose Brown (PO) - Ethan Gonzalez (Frontend) - Alejandro Saenz (Tech Lead)

Ashley Martinez (QA) - Marian Barba (Scrum Master / DevOps)

UTP - Calidad de Software - 2026 - Presentacion 10 minutos

DAILY SCRUM

Las 3 preguntas obligatorias del Daily

Que hicimos?

- 5 historias completadas (US-01 a US-05)
- Dashboard medico desplegado
- Seguridad RLS multicapa
- 65 tests automatizados PASS
- 20 bugs cazados y resueltos
- 5 agentes IA configurados

Que haremos en Sprint 2?

- Cancelar cita (autoservicio)
- Reprogramar cita a otro horario
- Tabla citas_historial (auditoria)
- Tests E2E con Playwright
- SonarCloud en CI
- Locust a 300 usuarios

Que bloqueos tuvimos?

- Bug B-20 race React (4 iteraciones)
- Detectado en uso real por PO
- Spinner >10s tras F5 (B-15+B-17)
- Resueltos con Plan Mode + tests
- Cero pendientes al cierre
- Documentados en GitHub

DEMO EN VIVO

Escanea el QR y pruebalo en tu celular



helios-web-production.up.railway.app

1. Registrate con un correo nuevo
2. Elige especialidad y sede
3. Selecciona medico de la lista
4. Elige horario en el calendario
5. Confirma con tu cedula panamena
6. Recibe codigo de cita unico

APORTE INDIVIDUAL

Que hizo cada integrante en el Sprint 1

Jose Brown

Product Owner

22 commits

- Validacion de cada deploy en produccion
- Detecto 8 bugs criticos en uso real
- Lidero las 4 iteraciones del B-20
- Configuro los 5 agentes IA
- Priorizo backlog del sprint

Marian Barba

Scrum Master / DevOps

26 commits

- Setup inicial del proyecto
- Deploys CI/CD a Railway
- Documentacion de metricas
- Coordinacion ceremonias Scrum
- Gestion del Kanban en Jira

Ethan Gonzalez

Dev Frontend

3 commits + paircoding

- Componentes React + TypeScript
- Fix B-05 (toolbar sticky)
- Fix B-08 (try/catch 5 hooks)
- Vista mobile del calendario
- Integracion Supabase Auth

Alejandro Saenz

Tech Lead

2 commits + arquitectura

- Arquitectura Supabase + RLS
- Funcion SQL crear_cita_atomica
- Fix B-09 (seguridad sessionStorage)
- 6 tablas + 3 vinculos auth
- Decisiones tecnicas del sprint

Ashley Martinez

QA Engineer

6 commits

- Pruebas funcionales del wizard
- Documentacion de metricas
- Casos de prueba para el demo
- Calculo 8.70 bugs/KLOC
- Validacion criterios aceptacion

FRONTEND

Lo que construimos en React + TypeScript + Vite

Stack frontend

- React 19	Componentes funcionales + hooks
- TypeScript 6	Tipado estricto, cero `any`
- Vite 8	Bundle 754 KB / 217 KB gzip
- Tailwind 3	Design system consistente
- React Router 7	SPA con ProtectedRoute
- FullCalendar 6	Vista calendario desktop

Componentes construidos

- SelectorEspecialidad	Lista filtrable con loading
- ListaMedicos	Cards con foto + datos
- CalendarioCitas	Slots disponibles tiempo real
- CalendarioCitasLista	Vista mobile (B-18)
- FormularioConfirmacion	Validacion cedula panamena
- ResumenCita	Codigo unico generado
- MedicoDashboard	Agenda hoy + historial
- ProtectedRoute	Guard por rol (B-13)

BACKEND / SQL

Arquitectura Supabase + Row Level Security

6 tablas + 3 vinculos auth

- sedes	3 policlinicas
- especialidades	5 especialidades
- medicos	20 cuentas demo
- horarios	1200+ slots disponibles
- pacientes	Datos personales + cedula
- citas	Tabla central + estados
- auth_paciente_link	Mapea auth.users -> pacientes
- auth_medico_link	Mapea auth.users -> medicos

Seguridad + funcion atomica

Row Level Security (RLS)

- paciente_ve_sus_citas: auth.uid = dueno
- medico_ve_sus_citas: auth.uid = asignado
- paciente_crea_su_link: WITH CHECK estricto
- Atacante con DevTools NO puede modificar ajeno

Funcion SQL atomica

crear_cita_atomica(paciente, horario)

- Reserva horario + crea cita en 1 transaccion
- Evita race: 2 pacientes mismo slot
- SECURITY DEFINER + validacion auth.uid
- Tested con Locust 200 usuarios concurrentes

FLUJO USUARIO

Como navega un paciente / medico por HELIOS

Rutas principales:

Paciente nuevo:

Registro -> Login -> Agendar

Paciente existente:

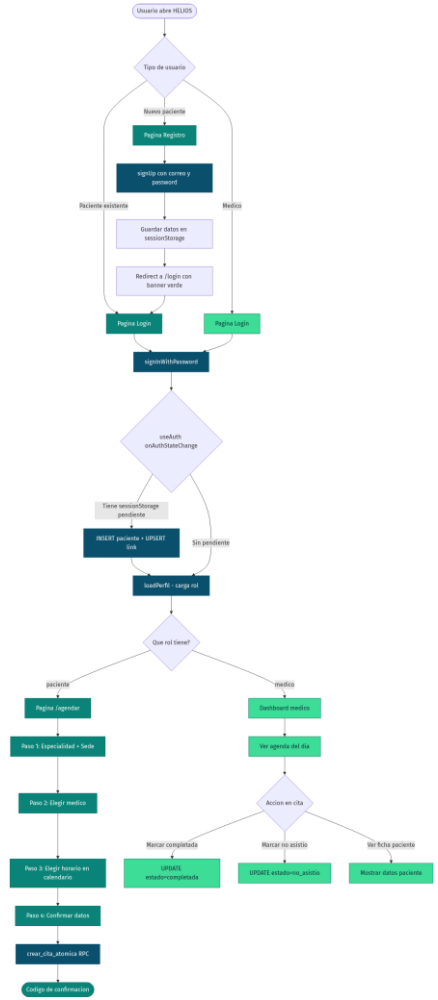
Login -> Agendar / Mis Citas

Medico:

Login -> Dashboard agenda hoy

Race React (B-20):

INSERT en onAuthStateChange



AGENTES IA

5 agentes IA especializados del equipo HELIOS

Cada agente esta especializado en una responsabilidad. Coordinacion automatica.

Team Orchestrator

helios-orchestrator

Coordina al equipo segun el cambio. SIEMPRE se invoca primero.

Cuando:

Decide que agentes lanzar y en que orden

Frontend Expert

helios-frontend

React 19 + TS 6 + Vite + Tailwind + Supabase

Cuando:

Conoce el stack exacto del proyecto

Bug Hunter

helios-bug-hunter

Detecta loading infinitos, race conditions, promesas sin catch

Cuando:

Invocar antes de cada deploy + en bugs reportados

Clean Code

helios-clean-code

JSDoc, naming, complexity, codigo muerto, agrupacion imports

Cuando:

Invocar tras fixes grandes o antes de merge

Test Engineer

helios-test-engineer

Vitest + RTL, cobertura >=60%, tests de regresion por bug

Cuando:

Invocar tras agregar feature o fix de bug

Resultado: ciclos Plan->Code->Review->Test->Deploy en <30 min cada bug

METRICAS ISTQB

Lo que medimos segun el estandar internacional

65

Tests automatizados

Vitest + RTL. Todos pasan.
Threshold 60% activo en CI.

61.4%

Cobertura de codigo

Statements 61.4% - Functions 90.9%.
Build falla si baja de 60%.

300 ms

Latencia P95

Locust 200 usuarios concurrentes.
Meta era 2s. Lo superamos 6.6x.

20

Defectos resueltos

Tasa 8.70 bugs/KLOC (rango profesional).
Cero pendientes al cierre.

BUGS DETECTADOS

20 bugs cazados y resueltos en uso real

Cada bug tiene ID, causa raiz, fix aplicado y test de regresion. Trazables en GitHub.

Distribucion por severidad

Critica	**7 bugs**	B-06, B-07, B-11, B-14, B-19, B-20.x
Alta	**8 bugs**	B-04, B-08, B-10, B-12, B-13, B-15, B-17, B-18
Media	**4 bugs**	B-01, B-05, B-09, B-16
Baja	**1 bug**	B-03

Quien detecto los bugs

PO en uso real:	**15 bugs (75%)**
Code review:	**3 bugs (15%)**
Tests automaticos:	**2 bugs (10%)**

CASO DESTACADO

B-20: race React closure

4 iteraciones hasta el fix definitivo

B-20:	Separar registro de login (signUp + sessionStorage)
B-20.1:	signOut tras signUp evita auto-login
B-20.2:	Polling JWT + errores visibles en pantalla
B-20.3:	FIX DEFINITIVO: mover INSERT a useAuth listener

La pista del PO fue clave:

"si cierro sesion y vuelvo a entrar, funciona"

SONARCLOUD LIVE

Analisis de calidad y seguridad en vivo

Vamos a abrir SonarCloud y mostrar:

- **Bugs detectados** — Issues encontrados automaticamente
- **Code smells** — Patrones que pueden causar problemas
- **Vulnerabilidades** — Problemas de seguridad
- **Coverage** — Codigo cubierto por nuestros tests
- **Duplication** — Codigo duplicado que refactorizar
- **Quality Gate** — Pasa o falla threshold de calidad

SonarCloud

sonarcloud.io/project/overview?id=MarianC-programing_Helios

DEMO EN VIVO

Abriremos el dashboard

y revisaremos los hallazgos

(escanea cada push automatico)

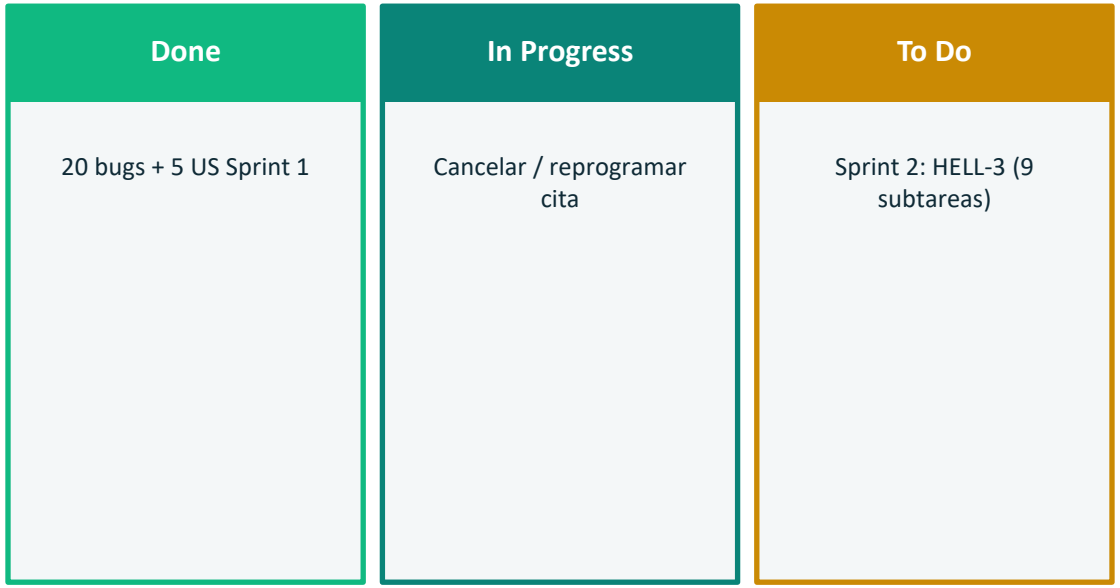
KANBAN + CIERRE

Estado del Sprint 1 + roadmap Sprint 2

7 herramientas usadas

- GitHub	Repositorio + commits firmados
- Jira	Tablero Kanban 21 issues
- Vitest	65 tests automatizados
- Locust	P95 300ms con 200 users
- Supabase	BD + Auth + API
- Railway	Deploy CI automatico
- SonarCloud	Calidad + seguridad (NUEVO)

Tablero Kanban Jira



MVP funcional - Metricas medidas - Proceso documentado - Agentes IA

Gracias. Preguntas?